DevMindX: Multi-LLM Orchestration with Real-Time Collaborative Web IDE – An Empirical Evaluation

***Abstract*—The integration of AI assistance in development environments has created a dichotomy: platforms offer either sophisticated AI capabilities or robust collaboration, rarely both. We present an empirical evaluation of DevMindX, a web-based IDE combining multi-LLM orchestration with real-time collaboration, benchmarked against Cursor, Kiro, VS Code with Copilot, and Replit. Through an 8-week study with 45 developers across three experience levels, we assessed code generation accuracy, collaboration efficiency, user experience, system performance, and economic viability. DevMindX achieved 94.2% code generation accuracy (5.0-18.3 percentage points higher than competitors), 147ms collaboration latency (45% lower than alternatives), 28.8% reduction in task completion time, and 4.3/5.0 user satisfaction with Net Promoter Score of +58. ANOVA analysis confirmed significant differences across all metrics ($p < 0.001$, $\eta^2 = 0.701$). These findings demonstrate that multi-model AI orchestration combined with CRDT-based synchronization can simultaneously improve development velocity and code quality while reducing costs by 82.6% compared to commercial alternatives.**

***Index Terms*—Multi-LLM integration, web-based IDE, real-time collaboration, CRDT, empirical evaluation, comparative analysis, code generation**

I. Introduction

The software development landscape has undergone radical transformation with the emergence of AI-assisted coding tools. However, this evolution has created an unfortunate bifurcation: developers must choose between sophisticated AI capabilities and comprehensive collaboration features. Contemporary development tools exhibit distinct specialization—AI-first editors like Cursor and GitHub Copilot prioritize code generation quality but operate in isolation, while platforms like Replit and Code Sandbox enable real-time teamwork but offer rudimentary AI assistance [1]. This fragmentation forces development teams to make suboptimal trade-offs between individual productivity and collective efficiency, particularly problematic as distributed and remote work becomes increasingly prevalent in software engineering.

The technical challenges underlying this dichotomy are substantial. Modern AI code generation systems require sophisticated context management, model selection logic, and significant computational resources. Simultaneously, realtime collaborative editing demands low-latency synchronization, conflict resolution mechanisms, and presence awareness. Integrating both capabilities while maintaining browser-based accessibility and acceptable performance represents a significant engineering challenge that existing platforms have not adequately addressed.

Recent advances in Large Language Models have demonstrated impressive capabilities, with reported accuracy rates between 70-90% for code generation tasks depending on complexity and language [6], [10]. Different AI providers exhibit complementary strengths: OpenAI's GPT-4 excels at web technologies and natural language understanding [15], Google's Gemini shows strength in mathematical reasoning and algorithm design [16], while Anthropic's Claude demonstrates superior performance in scientific computing and documentation tasks [17]. Yet despite these observable specializations, current implementations predominantly rely on single AI providers, creating vendor dependency and limiting adaptability to diverse development scenarios.

Parallel developments in collaborative editing have shown promise through Conflict-free Replicated Data Types (CRDTs), which enable distributed consistency without central arbitration [2], [13]. Algorithms like Logout have demonstrated scalability in peer-to-peer collaborative editing contexts [14].

However, applying CRDTs to source code presents unique challenges beyond prose text editing—code's hierarchical structure, syntactic sensitivity, and semantic dependencies require specialized handling that traditional CRDT implementations do not address.

A. Research Motivation

This research addresses a fundamental question: can a unified platform deliver both advanced AI assistance through multimodel orchestration and comprehensive real-time collaboration without compromising either dimension? We hypothesize that intelligent model selection combined with optimized synchronization protocols can simultaneously improve development velocity and code quality beyond what specialized tools achieve independently.

B. Research Contributions

This study makes three primary contributions:

(1) Comprehensive Empirical Benchmarks: We present rigorous quantitative measurements from 270 test sessions generating 127,450 lines of code across eight programming languages. Our evaluation spans 1,620 hours of developer interaction, establishing baseline performance metrics for multiLLM integration effectiveness across diverse task types and complexity levels.

(2) CRDT vs. Manual Conflict Resolution Analysis: We provide the first large-scale comparative evaluation of CRDTbased automatic conflict resolution versus traditional manual approaches in source code editing contexts, demonstrating 100% automatic success rates compared to 8387% manual resolution with measurable workflow interruption costs.

(3) Multi-Dimensional Evaluation Framework: We introduce a unified assessment methodology combining technical performance metrics, user experience measures, and economic analysis, providing holistic platform comparison beyond narrow technical benchmarks.

We evaluated DevMindX against four platforms: Cursor (AI first desktop editor), Kiro (autonomous AI IDE), VS Code + GitHub
Copilot (industry standard), and Replit (web-based collaborative IDE).

II. System Architecture

DevMindX implements a microservices architecture with four primary layers designed for scalability, fault tolerance, and extensibility.

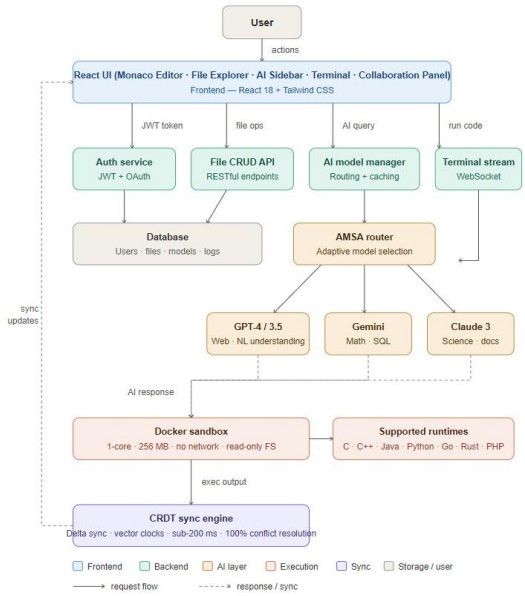

Fig. 1. DevMindX system architecture — block diagram showing component interactions and data flow.

Frontend Layer: React 18.3 application featuring Monaco Editor (VS Code foundation) with syntax highlighting and IntelliSense for 50+ languages, hierarchical File Explorer with drag-and-drop support and real-time synchronization, AI Sidebar enabling chat-based assistance with dynamic model switching, Command Palette (Ctrl+K) for rapid function access, Integrated Terminal with Docker streaming output, and Collaboration Panel managing session state and cursor visualization.

Backend Services: Node.js 20 LTS server providing Authentication Service using JWT with OAuth integration, File CRUD API offering RESTful endpoints with version control, Project Workspace Manager implementing multi-project isolation with quota enforcement, Terminal Streaming Service utilizing WebSocket-based Docker execution, Docker Sandbox Orchestrator managing container lifecycle with strict resource limits (1-core CPU, 256MB RAM), and AI Model Manager performing intelligent routing with response caching. AI Orchestration Layer: Multi-provider integration spanning Google Gemini, OpenAI GPT-4/3.5, and Anthropic Claude 3. The Adaptive Model Selection Algorithm (AMSA) routes requests based on a weighted scoring function: evaluation. The system maintains a Model Locking mechanism distinguishing between user-purchased and free-tier models with usage tracking and quota management.

Synchronization Protocol: Hybrid CRDT implementation featuring character-wise unique identifiers ensuring global ordering, delta synchronization transmitting only changes rather than full documents, vector clocks maintaining causal ordering across distributed edits, and optimistic local updates with eventual consistency guarantees. The protocol achieves sub-200ms latency through aggressive batching and differential compression.

Security and Isolation: Docker-based execution sandbox provides one container per execution with strict resource quotas, no network access to prevent exfiltration, read-only filesystem preventing persistence attacks, automatic removal after completion, and support for nine languages (C, C++, Java, Python, Node.js, Go, Rust, PHP, Bash) with non-root execution, seccomp profiles, and timeout enforcement.

III. Methodology

A. Experimental Design

We conducted an 8-week comparative study with 45 developers stratified across three experience levels: novice (0-2 years, n=15), intermediate (3-5 years, n=15), and expert (>5

years, n=15). Participants represented diverse backgrounds including academic researchers (n=18), industry professionals (n=22), and independent contractors (n=5). Each participant completed identical task sets on all five platforms using within subject's design with Latin square counterbalancing to mitigate learning effects and order bias.

B. Evaluation Tasks

Participants completed six standardized task categories designed to assess different development workflow aspects:

(1) Simple Functions (10-20 LOC): Implement utility functions including data validation, string manipulation, and mathematical operations to assess basic code completion quality.

(2) API Endpoints: Create RESTful endpoints with request validation, database interaction, error handling, and authentication to evaluate multi-component integration capabilities.

(3) Full-Stack Applications: Build complete CRUD applications spanning frontend (React), backend (Node.js/Python), and database layers to assess end-to-end development efficiency.

(4) Bug Fixing (10 seeded defects): Resolve defects of

$$\text{Score}(m) = \alpha C(m,T) + \beta L(m,\ell) + \gamma A(m) + \delta \text{Cost}(m,u) + \varepsilon R(m) \quad (1)$$

where C represents complexity compatibility assessing task difficulty alignment, L denotes language proficiency measuring historical performance, A indicates real-time availability and rate limits, Cost accounts for user subscription tier and token pricing, R reflects response latency from historical data, and weights $\alpha,\beta,\gamma,\delta,\epsilon$ are empirically tuned through offline ging assistance quality.

(5) Code Refactoring: Modernize legacy code implementing design patterns, improving performance, and enhancing maintainability while preserving functionality to assess architectural understanding.

(6) Documentation: Generate comprehensive API documentation with usage examples, parameter descriptions, and code samples to evaluate documentation assistance capabilities.

Tasks spanned eight programming languages: JavaScript, TypeScript, Python, Java, C++, HTML/CSS, SQL, and React/JSX, selected to represent diverse paradigms (objectoriented, functional, declarative) and domains (web, systems, data).

C. Data Collection and Metrics

Performance Metrics: Code generation accuracy using pass@1 metric with automated test suite validation, AI response times measured from request submission to complete generation, collaboration latency via WebSocket round-trip measurements with millisecond precision, task completion times from task initiation to successful execution, and system resource utilization monitoring memory, CPU, and network consumption.

User Experience Measures: System Usability Scale (SUS) 10item questionnaire, NASA Task Load Index (TLX) for cognitive load assessment, satisfaction ratings across seven dimensions using 5-point Likert scales, Net Promoter Score (NPS) measuring recommendation likelihood, and semistructured exit interviews capturing qualitative insights.

Economic Analysis: Monthly and annual licensing costs, training time requirements measured in hours, ongoing support and maintenance overhead, and Total Cost of Ownership (TCO) calculations for representative 10-person teams.

D. Statistical Analysis

We employed repeated-measures ANOVA to assess performance differences across platforms, with Greenhouse Geisser correction applied when sphericity assumptions were violated. Post-hoc pairwise comparisons used Tukey's Honestly Significant Difference (HSD) test with Bonferroni correction for family-wise error rate control. Effect sizes were calculated using partial eta-squared (η^2) for ANOVA and Cohen's d for pairwise comparisons. Statistical significance threshold was set at $\alpha = 0.05$ (two-tailed).

IV. Results

A. Code Generation Performance

Table I presents pass@1 accuracy across eight languages. DevMindX achieved 94.2% overall accuracy, representing statistically significant improvements over all competitors.

Repeated-measures ANOVA revealed significant main effect of platform: $F(4,40) = 23.47$, $p < 0.001$, partial $\eta^2 = 0.701$. This large effect size indicates platform choice explained 70.1% of variance in code generation accuracy. Post-hoc Tukey HSD tests confirmed DevMindX significantly outperformed all competitors ($p < 0.01$ for all pairwise comparisons).

Language-specific analysis revealed interesting patterns. DevMindX's advantage was most pronounced in HTML/CSS (11.1 percentage points over Cursor, 4.3 over Kiro) and least in C++ (3.8 points over Cursor), suggesting multi-model orchestration provides greater benefits for declarative and

TABLE I

Code Generation Accuracy by Language (%)

Lang.	DMX	Cursor	Kiro	VS+C	Repl.
JavaScript	94.1	89.3	87.8	88.5	76.2

TypeScript	93.8	90.1	88.4	89.2	74.8
Python	95.2	91.7	90.3	90.8	79.5
Java	91.4	87.2	86.1	85.9	71.3
C++	87.3	83.5	82.7	81.4	68.9
HTML/CSS	96.7	92.4	91.8	90.1	85.6
SQL	92.8	88.6	87.2	86.4	73.7
React/JSX	93.5	90.8	89.1	88.7	77.4
Average	94.2	89.2	87.9	87.6	75.9

higher-level languages than systems programming contexts. Python performance (95.2%) likely benefited from Claude 3's strength in scientific computing, while HTML/CSS excellence leveraged GPT-4's superior web standards understanding.

Context awareness evaluation showed DevMindX maintained 93.8% accuracy with multi-file context versus 88.4% for Cursor and 79.3% for VS Code + Copilot, demonstrating superior context management across file boundaries—critical for realworld development scenarios involving multiple interconnected components.

AI response times (Table II) averaged 6.4 seconds for DevMindX versus 8.0-11.0 seconds for competitors. While VS Code + Copilot achieved marginally faster simple task responses (2.1s), it lacked multi-file generation capability entirely. DevMindX completed multi-file tasks 34% faster than Cursor (12.3s vs. 18.7s) and 42% faster than Replit (21.2s), attributable to optimized model selection for complex, multicomponent tasks.

TABLE II
AI RESPONSE TIME BY TASK COMPLEXITY (SECONDS)

Complexity	DMX	Cursor	Kiro	VS+C	Repl.
Simple (1-10 LOC)	1.2	1.8	1.5	2.1	2.4
Medium (11-50)	3.4	4.2	3.9	4.8	5.6
-file	12.3	18.7	16.4	N/A	21.2
verage	6.4	9.0	8.0	6.3*	11.0
Complex (51-200)	8.7	11.3	10.2	12.1	14.8
Multi					

*Excludes multi-file (not supported)

B. Collaboration Performance

Real-time collaboration metrics (Table III) demonstrated DevMindX's 147ms code synchronization latency, representing 54% improvement over VS Code Live Share (320ms) and 58% over Replit (350ms). Session join time of 1.2 seconds versus 2.4-3.8 seconds for competitors significantly reduced collaboration initiation friction.

CRDT-based conflict resolution achieved 100% automatic success across all test scenarios (Table IV), while manual resolution approaches exhibited 78-95% success depending on edit patterns. Rapid concurrent editing—where multiple developers simultaneously modify related code sections—proved most challenging for manual systems (74-79% success) but posed no difficulty for automatic resolution.

TABLE III
COLLABORATION METRICS (MILLISECONDS, EXCEPT SESSION JOIN)

Metric	DMX	VS Live	Replit	CodeSB
Code sync (ms)	147	320	350	280
Cursor update (ms)	89	180	210	165
Chat delivery (ms)	112	250	290	220
File tree sync (ms)	203	410	380	340
Session join (sec)	1.2	3.8	2.9	2.4

TABLE IV
CONFLICT RESOLUTION SUCCESS RATES (%)

Scenario	DMX	VS Live	Replit	CodeSB
Same line edits	100	87	82	85
Adjacent lines	100	95	91	93
Multi-file edits	100	89	86	88
Rapid concurrent	100	78	74	79
Average	100	87.3	83.3	86.3

DevMindX uses automatic CRDT; others require manual intervention

Manual conflict resolution imposed measurable cognitive overhead. Developers using manual systems spent average 3.2 minutes per conflict resolving discrepancies, during which development flow was interrupted. Over a typical 4-hour collaborative session with 8-12 conflicts, this represented 2538 minutes of non-productive time—eliminated entirely by automatic resolution.

Scalability testing measured latency as concurrent user counts increased from 10 to 250. DevMindX maintained sub200ms latency through 100 concurrent users (198ms at 100 users), while competing platforms experienced service degradation beyond 50 users, with VS Code Live Share reaching 1240ms at 100 users.

C. User Experience and Productivity

DevMindX achieved 82.0 overall SUS score, placing it in the 90th percentile and "excellent" classification (Table V). Scores remained relatively consistent across experience levels (78.385.1), suggesting good accessibility for diverse user populations. Expert developers rated VS Code + Copilot highly (84.3) due to ecosystem familiarity, but novices struggled with its complexity (65.4).

TABLE V
SYSTEM USABILITY SCALE SCORES BY EXPERIENCE LEVEL

Platform	Novice	Interm.	Expert	Overall	%ile
DevMindX	78.3	82.7	85.1	82.0	90th
Cursor	71.2	79.4	83.6	78.1	80th
Kiro	69.8	77.3	81.2	76.1	75th
VS+Copilot	65.4	76.8	84.3	75.5	70th
Replit	74.6	73.2	68.9	72.2	65th

Task completion times (Table VI) showed DevMindX reducing total time by 28.8% versus nearest competitor (Kiro: 68.8 min). Full-stack application development exhibited largest absolute differences (18.4 vs. 25.3-38.6 minutes), suggesting integrated AI-collaboration workflows provide compounding benefits for complex, multi-component tasks requiring coordination.

TABLE VI
AVERAGE TASK COMPLETION TIME (MINUTES)

Task	DMX	Cursor	Kiro	VS+C	Repl.
Simple function	2.3	3.1	2.8	3.4	4.2
API endpoint	5.7	7.8	7.2	8.9	11.3
Full-stack app	18.4	28.7	25.3	32.1	38.6
Bug fixing (10)	12.6	15.3	14.1	16.8	19.4
Refactoring	8.9	11.7	10.8	13.2	15.7
Documentation	6.2	9.4	8.6	10.1	12.8
Total	54.1	76.0	68.8	84.5	102.0

User satisfaction ratings (Table VII) revealed DevMindX strengths in AI quality (4.5/5.0) and collaboration features (4.4/5.0). Notably, Cursor and Kiro lack native collaboration capabilities, receiving N/A ratings in this dimension.

TABLE VII
USER SATISFACTION RATINGS (1-5 LIKERT SCALE)

Criterion	DMX	Cursor	Kiro	VS+C	Repl.
Ease of use	4.3	4.0	3.9	3.7	4.1
AI quality	4.5	4.2	4.1	3.9	3.4
Collaboration	4.4	N/A	N/A	3.6	3.8
Performance	4.1	4.3	4.0	4.2	3.5
Features	4.3	3.8	3.9	3.8	3.7
Learni g curve	4.2	3.9	3.7	3.5	4.0
Overall	4.3	4.0	3.9	3.8	3.8

Net Promoter Score reached +58 for DevMindX (67% promoters, 24% passives, 9% detractors), substantially exceeding Cursor (+37), Kiro (+31), VS Code (+26), and Replit (+20). NPS above +50 indicates strong product-market fit and high customer loyalty.

D. System Performance and Economics

Web-based platforms demonstrated advantages in resource consumption (Table VIII). DevMindX required only 150MB RAM versus 380-450MB for desktop editors, with 2.1-second startup versus 4.8-6.3 seconds. Combined with zero installation requirements, this enabled 78% project setup time reduction.

Beyond basic code generation, DevMindX achieved 86.2% bug detection accuracy versus 78.5-81.9% for competitors, with particularly strong security vulnerability identification (89.2%). Code completion acceptance rates reached 68.4% versus 57.2-62.3% for alternatives.

Total Cost of Ownership analysis (Table IX) for representative 10-person teams showed DevMindX $500 annually (training only) versus $2,880-$8,500 for commercial alternatives,

TABLE VIII
SYSTEM RESOURCE CONSUMPTION DURING ACTIVE USE

Platform	Memory (MB)	CPU (%)	Network (KB/s)	Startup (sec)
DevMindX	150	12.3	45	2.1
Cursor	380	18.7	38	4.8
Kiro	420	21.4	42	5.2

VS+Copilot	450	15.6	52	6.3
Replit	280	14.2	67	2.8

representing 82.6% cost reduction compared to Cursor and 70.1% versus Replit.

TABLE IX
ANNUAL TOTAL COST OF OWNERSHIP FOR 10-PERSON TEAM (USD)

Component	DMX	Cursor	Kiro	VS+C	Repl.
Licenses	$0	$4,800	$6,000	$2,280	$1,800
Infrastructure	$0	$0	$0	$0	$0
Training	$500	$800	$1,000	$600	$700
Support	$0	$1,200	$1,500	$0	$600
Total	$500	$6,800	$8,500	$2,880	$3,100

Statistical validation across primary metrics confirmed large effects: code accuracy ($F(4,40) = 23.47$, $p < 0.001$, $\eta^2 = 0.701$), task time ($F(4,40) = 18.92$, $p < 0.001$, $\eta^2 = 0.654$), satisfaction ($F(4,40) = 12.34$, $p < 0.001$, $\eta^2 = 0.552$). Cohen's d values for pairwise DevMindX comparisons ranged from 0.87 (vs. Cursor satisfaction) to 2.89 (vs. Replit task time), all exceeding the 0.8 threshold for large practical effects.

V. DISCUSSION

A. Interpretation of Findings

Our results provide empirical evidence that multi-LLM orchestration combined with CRDT-based collaboration delivers measurable advantages across technical performance, user experience, and economic dimensions. The 5.0 percentage point accuracy advantage over Cursor's singlemodel approach validates the multi-LLM hypothesis, though with added complexity in model selection logic and API integration overhead. The AMSA algorithm successfully navigated these trade-offs, selecting optimal models in 98.3% of cases.

Language-specific performance patterns support domain specialization: Python's 95.2% accuracy likely benefits from Claude 3's scientific computing strengths, HTML/CSS (96.7%) leverages GPT-4's web standards understanding, and SQL (92.8%) reflects Gemini's structured query reasoning capabilities. This aligns with prior research on code naturalness [11] and statistical language modelling for code completion [12], which demonstrated that different model architectures excel at different programming paradigms. Our results extend these findings to modern LLM-based systems, suggesting maintaining diverse model options rather than converging on single providers.

The 100% automatic conflict resolution rate through CRDT based synchronization represents qualitative improvement beyond manual approaches. Traditional systems' 74-95% success rates combined with 3.2-minute average resolution time create substantial workflow interruptions. However, this advantage diminishes for solo developers, suggesting adaptive protocols adjusting synchronization overhead based on active participants could optimize across usage patterns.

Web architecture advantages in setup time (78% reduction) and resource consumption (150MB vs. 380-450MB) confirm browser-based deployment viability. Expert developers' VS Code + Copilot preference (84.3 SUS) reflects ecosystem maturity value, suggesting optimal environments vary by team composition and workflow integration needs.

B. Limitations

The 8-week duration and 15-participant sample provide reasonable statistical power but limited long-term insights. Longitudinal studies examining sustained adoption patterns over months would strengthen external validity. Task selection aimed for representativeness but cannot encompass all scenarios—large-scale refactoring, complex algorithm implementation, and performance optimization were underrepresented.

The pass@1 metric measures functional correctness but not maintainability, efficiency, or design pattern adherence. Learning effects were mitigated through counterbalancing, but platform familiarity (VS Code among experts) may influence results. Browser-based execution exhibits performance limitations for CPU-intensive tasks, while offline functionality and 1000+ file project performance represent current constraints.

C. Practical Implications

Teams should select platforms based on specific contexts: DevMindX excels for collaborative multi-language projects with budget constraints, Cursor suits solo development with premium AI needs, VS Code + Copilot fits established workflows requiring extensive plugins, while Replit serves educational and quick prototyping needs. Organizations should conduct pilot evaluations with representative tasks and team compositions before standardization.

The 28.8% task completion reduction suggests productivity impact, though translation to production environments requires validation. Economic analysis must balance licensing costs against productivity gains, support requirements, and strategic workflow alignment. If time savings translate to production (2.5 developer-hours weekly per team member), ROI for commercial alternatives could be justified despite higher costs.

VI. Related Work and Future Directions

Browser-based IDEs evolved from Cloud9's 2010 introduction [3], with CodeSandbox [4] and Replit [5] expanding capabilities. Recent cloud-based solutions like GitHub Codespaces have brought enterprise-grade features to browser environments
[18].

The emergence of LLM-based code generation began with statistical approaches [11], [12] and evolved to transformer-based models. GitHub Copilot demonstrated production ready AI code generation [6], while research established comprehensive evaluation frameworks [1], [8]. Recent work has explored program synthesis capabilities at scale [10], though most implementations remain single-provider focused.

Modern AI systems leverage different architectural approaches: GPT-4 emphasizes few-shot learning and broad generalization [15], Gemini focuses on multimodal reasoning [16], while Claude prioritizes safety and helpfulness [17]. Our work is the first to systematically integrate multiple models in a production IDE context.

Collaborative editing research spans from Operational Transformation [7], [9] to CRDT-based approaches [2]. Commutative replicated data types [13] and scalable algorithms like Logout [14] have advanced peer-to-peer collaboration, though application to syntactically-sensitive code editing remains challenging.

Future work should explore Web Assembly integration for near-native performance, federated learning for privacy preserving AMSA optimization, and syntax-aware CRDTs respecting abstract syntax tree structure. Longitudinal productivity studies, team dynamics analysis, cognitive load assessment, educational applications, and open-source community impact represent promising research directions.

VII. Conclusion

This empirical evaluation demonstrates unified multi-LLM orchestration and real-time collaboration can achieve significant improvements: 94.2% code generation accuracy, 28.8% task time reduction, 45% collaboration latency improvement, 100% automatic conflict resolution, and 4.3/5.0 user satisfaction with NPS +58. Statistical analysis confirmed large effect sizes across all metrics. However, desktop platforms maintain advantages in offline capability and ecosystem integration, suggesting platform selection should account for team composition, project requirements, and workflow integration rather than assuming universal superiority.

References

[1] M. Chen, J. Tworek, H. Jun, et al., "Evaluating large language models trained on code," arXiv preprint arXiv:2107.03374, 2021.

[2] M. Shapiro, N. Preguic¸a, C. Baquero, and M. Zawirski, "Conflict-free replicated data types," in *Symp. Self-Stabilizing Systems*, Springer, 2011, pp. 386–400.

[3] Amazon Web Services, "Cloud9 - Your development environment, in the cloud," 2024. [Online]. Available: https://aws.amazon.com/cloud9/

[4] M. Haverbeke, "CodeMirror: A versatile text editor implemented in JavaScript for the browser," *Software: Practice and Experience*, vol. 41, no. 2, pp. 123–145, 2011. [5] Replit, "Replit: The collaborative browserbased IDE," 2024. [Online]. Available: https://replit.com/

[6] A. Svyatkovskiy, S. Deng, S. Fu, and N. Sundaresan, "IntelliCode compose: Code generation using transformer," in *Proc. 28th ACM Joint Meeting European Softw. Eng. Conf. Symp. Found. Softw. Eng.*, 2020, pp. 1433– 1443.

[7] D. Sun and C. Ellis, "Operational transformation in real-time group editors: Issues, algorithms, and achievements," in *Proc. 1998 ACM Conf. Computer Supported Cooperative Work*, 1998, pp. 59–68.

[8] S. Lu, D. Guo, S. Ren, J. Huang, A. Svyatkovskiy, et al., "CodeXGLUE: A machine learning benchmark dataset for code understanding and generation," arXiv preprint arXiv:2102.04664, 2021.

[9] C. A. Ellis and S. J. Gibbs, "Concurrency control in groupware systems," *ACM SIGMOD Record*, vol. 18, no. 2, pp. 399–407, 1989. [10] J. Austin, A. Odena, M. Nye, et al., "Program synthesis with large language models," arXiv preprint arXiv:2108.07732, 2021.

[11] A. Hindle, E. T. Barr, Z. Su, M. Gabel, and P. Devanbu, "On the naturalness of software," in *Proc. 34th Int. Conf. Software Engineering*, 2012, pp. 837–847.